Product Requirements Document

AI Task Management Agent

ENVIRONMENT: ANTIGRAVITY IDE (FULL-STACK)

STATUS: DRAFT / PLANNING

VERSION: 2.0 (UPDATED ARCHITECTURE)

1. Executive Summary

The AI Task Management Agent is an intelligent productivity tool designed to automate, organize, and optimize user workflows. Unlike standard to-do list applications, this agent leverages Large Language Models (LLMs) to understand natural language intent, break down complex projects into actionable steps, and dynamically prioritize tasks. This project will be developed natively within the Antigravity IDE, adopting a decoupled full-stack architecture featuring a responsive web frontend and a dedicated Python-based AI logic server.

2. Project Objectives

- **Frictionless Input:** Allow users to add tasks using conversational language (e.g., "Remind me to prep for the marketing review next Tuesday").
- **Intelligent Breakdown:** Automatically parse large goals into structured, hierarchical sub-tasks.
- **Dynamic Prioritization:** Continuously re-rank user queues based on changing deadlines and identified blockers.
- **Contextual Awareness:** Learn from user completion patterns to suggest better time allocations.

3. Architecture & Tech Stack

Based on modern AI agent repository structures, the project will be divided into distinct frontend and backend environments:

- **Frontend UI Environment:** The user interface will be a modern web application built using Vite (`vite.config.js`) for rapid development and bundling. Styling will be handled via Tailwind CSS (`tailwind.config.js`, `postcss.config.js`). Core frontend logic will be managed through standard JavaScript and HTML files within the `ai-task-management-ui/` and `frontend/` directories.
- **Backend & Agent Environment:** The core logic engine will be written in Python (`requirements.txt`). The architecture separates concerns into distinct modules:
  - `api_server.py`: Manages endpoints and communication between the frontend and the AI logic.
  - `agent.py`: Houses the core LLM execution loop and prompt management.
  - `tools.py`: Contains definitions for external tools the agent can call (e.g., calendar sync, file parsing).
  - `database.py`: Handles local storage and schema management for tasks.
- **Execution & Testing:** The project will include startup batch scripts (`start.bat`, `start_api.bat`) for initializing the environment locally, alongside smoke tests (`smoke_test.py`) to verify agent health. An

4. Functional Requirements

Feature	Description	Priority
NLP Parsing	Interpret unstructured text input to extract title, dates, assignees, and context.	P0 - Critical
Auto-Decomposition	Agent generates 3-5 logical sub-tasks when given a broad overarching project command.	P0 - Critical
Smart Scheduling	Suggest optimal time blocks for tasks by cross-referencing estimated duration with user availability.	P1 - High
Status Aggregation	Provide a daily morning briefing summarizing overdue tasks, high-priority items, and schedule limits.	P1 - High
Third-Party Sync	Two-way synchronization with standard calendar protocols (CalDAV/Google Calendar).	P2 - Medium

5. AI-Specific Capabilities

Intent Recognition Engine: The core logic within `agent.py` will route user input through an intent classifier to determine if a prompt is a creation event, a modification event, or a query (e.g., "What should I work on next?").

Adaptive Reinforcement: The agent must log instances where users ignore its prioritization suggestions, using this feedback loop to adjust its weighting algorithm for future schedule generation.

6. Implementation Milestones

Phase	Deliverables	Target Timeline
Phase 1: Foundation	Initialize UI scaffolding using Vite and Tailwind. Establish the basic Python backend (`api_server.py`, `database.py`) and secure API key management via `.env` files. Create startup batch scripts.	Weeks 1 - 2
Phase 2: Core AI Loop	Implementation of `agent.py` and the NLP parsing engine. Enable basic task creation via chat and manual CRUD operations connecting the frontend to the API server.	Weeks 3 - 5
Phase 3: Intelligence	Populate `tools.py` with task auto-decomposition logic, priority sorting algorithms, and context-aware reminders.	Weeks 6 - 8
Phase 4: Polish & Launch	Calendar integration, execution of `smoke_test.py` for performance optimization, user acceptance testing, and finalizing deployment documentation.	Weeks 9 - 10

7. Success Metrics

TARGET TASK PARSING ACCURACY

95%

Correct extraction of date, time, and intent from raw user text input via `agent.py`.

TARGET API LATENCY

< 1.5s

Maximum round-trip time from user command submission to UI update from `api_server.py`.

8. Sign-Off & Next Steps

Upon approval of this PRD, the engineering team will initialize the core repository within the Antigravity IDE and commence Phase 1 architectural planning, generating the necessary UI and backend boilerplate files.